

ФГАОУ ВО «НИУ ИТМО»  
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ  
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

## Лабораторная работа №1

### Аппроксимация и интерполяция

(по дисциплине «Вычислительная математика»)

**Выполнил:**

Лабин Макар Андреевич,  
ВЫЧМ 3.2, Р3231, 466449.

**Проверила:**

Перл Ольга Вячеславовна



г. Санкт-Петербург, Россия  
2026

# Содержание

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Постановка задач</b>                               | <b>1</b>  |
| <b>2</b> | <b>Выполнение работы</b>                              | <b>2</b>  |
| 2.1      | Описание метода . . . . .                             | 2         |
| 2.1.1    | Интерполяционный многочлен в форме Лагранжа . . . . . | 2         |
| 2.1.2    | Многочлен Чебышёва . . . . .                          | 3         |
| 2.2      | Блок-схема численного метода . . . . .                | 4         |
| 2.3      | Листинг численного метода . . . . .                   | 6         |
| 2.4      | Результаты работы программы . . . . .                 | 7         |
| <b>3</b> | <b>Заключение</b>                                     | <b>11</b> |
| <b>4</b> | <b>Источники литературы</b>                           | <b>13</b> |

# 1 Постановка задач

Схема интерполяции в общем виде выглядит следующим образом:

$$f(t) \xrightarrow{\mathbf{R}} \{f_n\}_{n=0}^N \xrightarrow{\mathbf{I}} F(t)$$

Здесь  $f(t)$  — исходная функция,  $F(t)$  — интерполирующая.

Как видно из схемы, на исходную функцию действует оператор сужения  $\mathbf{R}$  на сетку  $\{t_n\}_{n=0}^N \subset [a, b]$ . Из неё формируется сеточная проекция функции  $f_n = \{f(t_n)\}_{n=0}^N$ . Затем происходит неоднозначное восстановление исходной функции, которое определяется оператором интерполяции  $\mathbf{I}$ .

В варианте №9 интерполяционные узлы определяются многочленами Чебышёва, а оператор интерполяции — интерполяционным многочленом в форме Лагранжа.

То есть в ходе лабораторной работы будут выполнены следующие задачи:

1. Разработка программного модуля для построения интерполяционного многочлена в форме Лагранжа  $L_n(t)$ .
2. Использование корней многочлена Чебышёва в качестве узлов интерполяции на заданном отрезке  $[a, b]$ .
3. Реализация итерационного процесса увеличения количества узлов  $n$  до достижения заданной точности с критерием останова:

$$|L_n(x) - L_{n-1}(x)| < \varepsilon, \text{ где } \varepsilon = 0.01$$

4. Вычисление значения интерполирующей функции в заданной точке  $x$  для различных входных функций и интервалов.

## 2 Выполнение работы

### 2.1 Описание метода

#### 2.1.1 Интерполяционный многочлен в форме Лагранжа

Рассмотрим произвольный многочлен степени  $N$ :

$$L_N(t) = \sum_{n=0}^N a_n t^n$$

Чтобы он был интерполяционным, необходимо условие:

$$L_N(t_n) = f_n, \quad n = \overline{0, N}$$

Оно равносильно решению СЛАУ:

$$\begin{cases} a_0 + a_1 t_0 + \dots + a_N t_0^N = f_0 \\ a_0 + a_1 t_1 + \dots + a_N t_1^N = f_1 \\ \dots \\ a_0 + a_1 t_N + \dots + a_N t_N^N = f_N \end{cases}$$

Определим совместность СЛАУ:

$$\Delta = \begin{vmatrix} 1 & t_0 & t_0^2 & \dots & t_0^N \\ 1 & t_1 & t_1^2 & \dots & t_1^N \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & t_N & t_N^2 & \dots & t_N^N \end{vmatrix} = \prod_{i \neq j}^N (t_i - t_j), \quad 0 \leq j < i \leq N$$

Матрица невырождена, если узлы интерполяции попарно различны. Допустим, это так. Значит, решение есть и оно единственно.

По методу Крамера найдём искомый многочлен:

$$L_N(t) = \sum_{n=0}^N \frac{\Delta_n}{\Delta} \cdot t^n$$

Хотя интерполирующая функция найдена, для представления в форме Лагранжа разложим  $\Delta_n$  по элементам  $n$ -го столбца:

$$a_n = \frac{\sum_{i=0}^N f(t_i) \Delta_{ni}}{\Delta}$$

Перегруппируем слагаемые, чтобы собрать члены с одинаковыми  $f(t_n)$ :

$$L_N(t) = \sum_{n=0}^N f(t_n) \cdot \Phi_n(t)$$

Здесь в качестве  $\Phi_n(t)$  обозначены некоторые линейные комбинации  $\{t^i\}_{i=0}^N$ . Можно заметить, что они обладают свойством:

$$\Phi_i(t_j) = \delta_i^j,$$

где  $\delta_i^j$  — символ Кронекера.

Из этого следует, что многочлены имеют вид:

$$\Phi_i(t) = A(t - t_0)(t - t_1) \dots (t - t_{i-1})(t - t_{i+1}) \dots (t - t_n)$$

Зная, что  $\Phi_i(t_i) = 1$ , имеем:

$$\begin{aligned} 1 &= A(t_i - t_0)(t_i - t_1) \dots (t_i - t_{i-1})(t_i - t_{i+1}) \dots (t_i - t_n) \implies \\ \implies \Phi_i(t) &= \frac{(t - t_0)(t - t_1) \dots (t - t_{i-1})(t - t_{i+1}) \dots (t - t_n)}{(t_i - t_0)(t_i - t_1) \dots (t_i - t_{i-1})(t_i - t_{i+1}) \dots (t_i - t_n)} \end{aligned}$$

Таким образом, мы вывели формулу для *интерполяционного многочлена в форме Лагранжа*:

$$L_N(t) = \sum_{n=0}^N f(t_n) \cdot \prod_{\substack{j=0 \\ n \neq j}}^N \frac{t - t_j}{t_n - t_j}$$

### 2.1.2 Многочлен Чебышёва

Функция вида

$$T_n(t) = \cos(n \arccos t), \quad t \in [-1, 1], \quad n \in \mathbb{N}_0$$

называется **многочленом Чебышёва первого рода**.

Доказательство факта, что приведённая функция является многочленом степени  $n$ , выполняется методом математической индукции.

Найдём нули функции  $T_n(t)$ :

$$\begin{aligned} \cos(n \arccos t) = 0 &\implies n \arccos t_k = \frac{\pi}{2} + \pi k, \quad k \in \mathbb{Z} \implies \\ \implies \arccos t_k &= \frac{2k+1}{2n} \pi \implies t_k = \cos \left( \frac{2k+1}{2n} \pi \right), \quad \frac{2k+1}{2n} \pi \in [0, \pi] \end{aligned}$$

Поскольку это многочлен степени  $n$ , он имеет не более  $n$  различных вещественных корней (*основная теорема алгебры*). Из  $\cos: \mathbb{R} \rightarrow \mathbb{R}$  следует, что все  $n$  корней лежат на отрезке  $[-1, 1]$ :

$$t_k = \cos \left( \frac{2k-1}{2n} \pi \right), \quad k = \overline{1, n}$$

Масштабируем корни на произвольный отрезок  $[a, b]$ :

$$t_k = \frac{a+b}{2} + \frac{b-a}{2} \cos \left( \frac{2k-1}{2n} \pi \right)$$

Таким образом, мы получили набор интерполяционных узлов  $\{t_n\}_{n=0}^N \subset [a, b]$  и можем приступить к составлению численного метода.

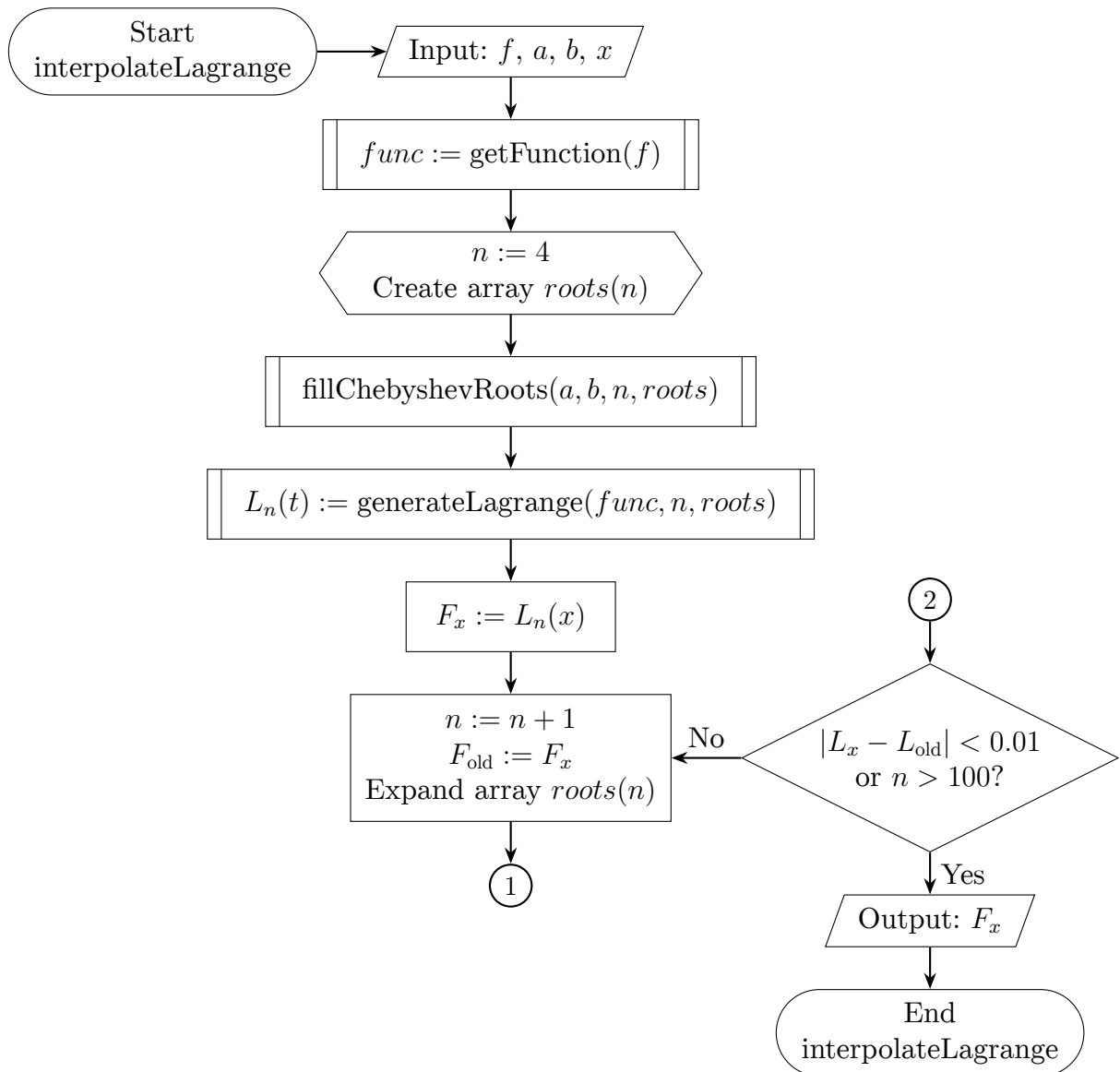
## 2.2 Блок-схема численного метода

На вход реализуемого метода подаются следующие данные:

- $f$  — номер интерполируемой функции; является целым числом.
- $a$  — левая граница интерполируемого интервала; является вещественным числом двойной точности.
- $b$  — правая граница интерполируемого интервала; является вещественным числом двойной точности.
- $x$  — значение аргумента для интерполяционного многочлена; является вещественным числом двойной точности.

На выходе метода ожидается значение интерполяционного многочлена в точке  $x$ , которое является вещественным числом двойной точности.

Далее приводятся блок-схемы, реализующие численный метод.



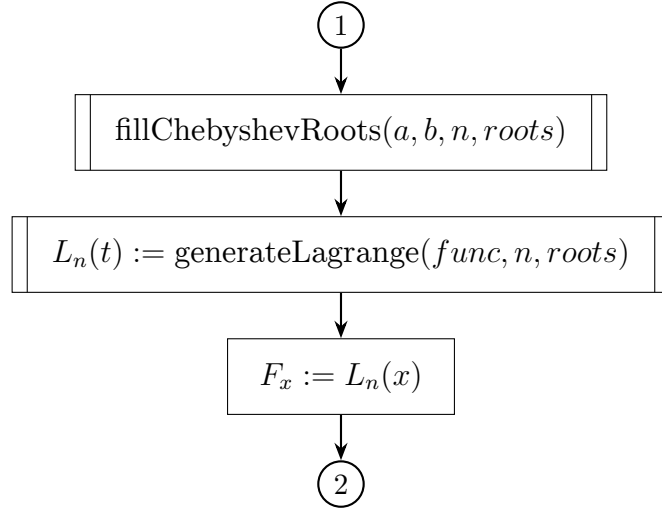
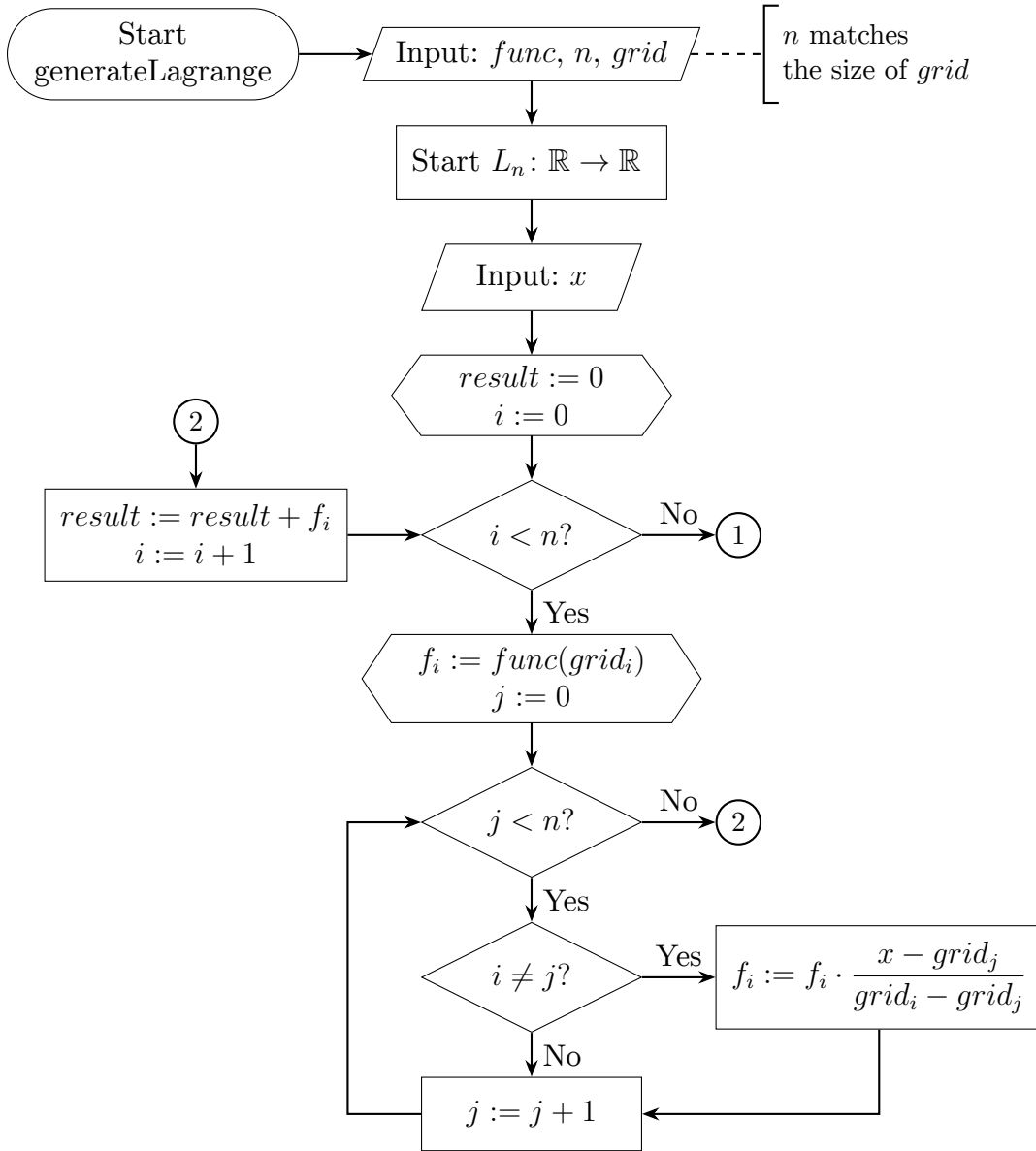


Схема 1 — Вычисление значения интерполирующей функции  $F(x)$  заданной точности.



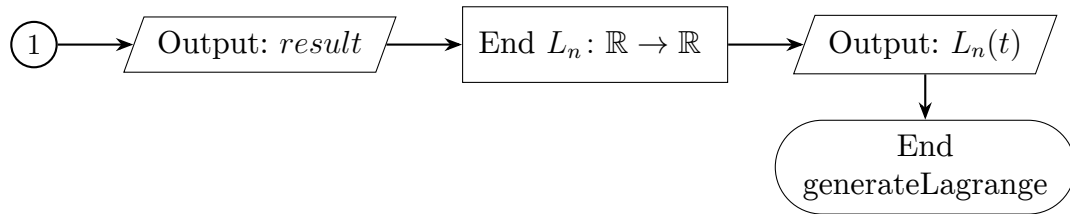


Схема 2 — Построение интерполяционного многочлена в форме Лагранжа  $L_n(t)$ .

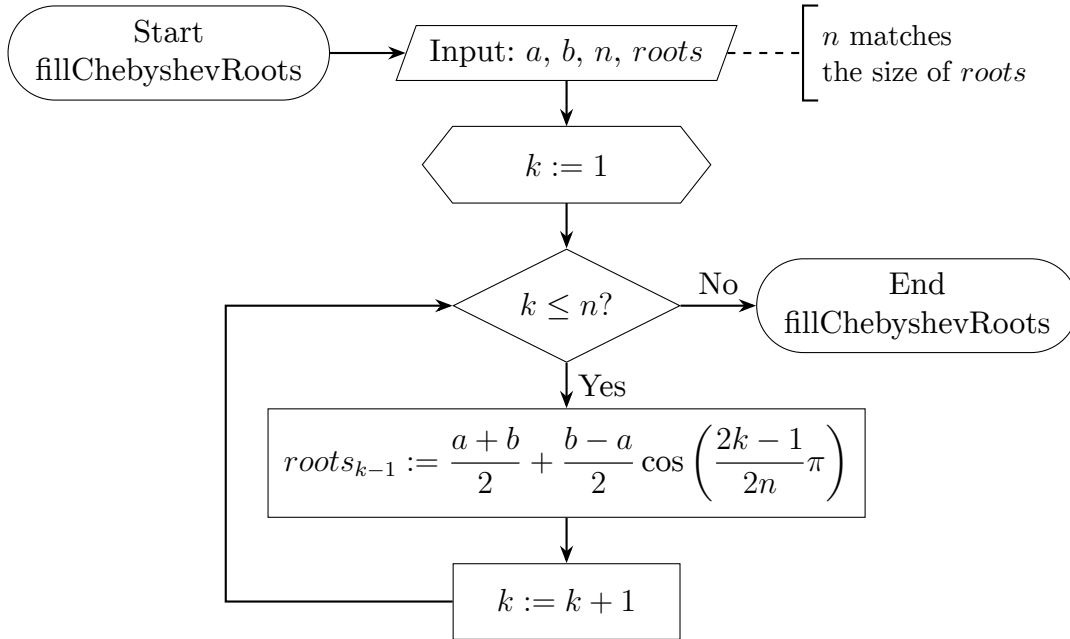


Схема 3 — Поиск корней многочлена Чебышёва первого рода  $T_n(t)$ .

## 2.3 Листинг численного метода

Для реализации блок-схем воспользуемся языком программирования C++.

```

1 double interpolate_by_lagrange(int f, double a, double b, double
  x) {
2   function<double(double)> func = get_function(f);
3   int n = 4;
4   std::vector<double> roots(n);
5   fill_chebyshev_polynomial_roots(n, a, b, roots.data());
6   double lagrange_old, lagrange_current =
    generate_lagrange_polynomial(n, func, roots.data())(x);
7   do {
8     n++;
9     lagrange_old = lagrange_current;
10    roots.reserve(n);
11    fill_chebyshev_polynomial_roots(n, a, b, roots.data());
12    lagrange_current = generate_lagrange_polynomial(n, func,
    roots.data())(x);
13  } while (abs(lagrange_current - lagrange_old) > 0.01 && n <
    100);
14  return lagrange_current;
  
```



15 }

Листинг 1 — Вычисление значения интерполирующей функции  $F(x)$  заданной точности.

```
1 function<double(double)> generate_lagrange_polynomial(const int
  n, const function<double(double)> func, const double grid[])
  {
2   return [n, func, grid](double x) {
3     double result = 0;
4     for (int i = 0; i < n; i++) {
5       double current_term = func(grid[i]);
6       for (int j = 0; j < n; j++) {
7         if (j != i) {
8           current_term *= x - grid[j];
9           current_term /= grid[i] - grid[j];
10        }
11      }
12      result += current_term;
13    }
14    return result;
15  };
16 }
```

Листинг 2 — Построение интерполяционного многочлена в форме Лагранжа  $L_n(t)$ .

```
1 void fill_chebyshev_polynomial_roots(const int n, const double a
  , const double b, double roots[]) {
2   for (int k = 1; k <= n; k++) {
3     roots[k - 1] = 0.5 * (a + b + (b - a) * cos(M_PI * (2 * k -
4       1) / (2.0 * n)));
5   }
6 }
```

Листинг 3 — Поиск корней многочлена Чебышёва первого рода  $T_n(t)$ .

## 2.4 Результаты работы программы

Поскольку отчёт верстается в  $\text{\LaTeX}$ , воспользуемся языками PGF/TikZ для построения графиков функций в векторной форме.

На приведённых далее графиках пунктиром обозначены интерполируемые функции  $f(x)$ , а чёрными точками — значения интерполирующей функции при данном аргументе  $x$ .

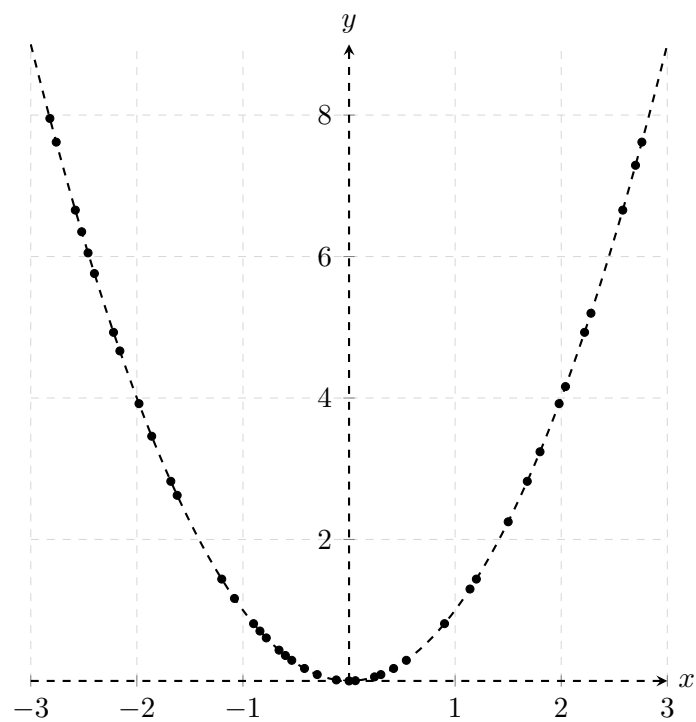


Рис. 1 — Интерполяция  $f(x) = x^2$  на  $[-3, 3]$ .

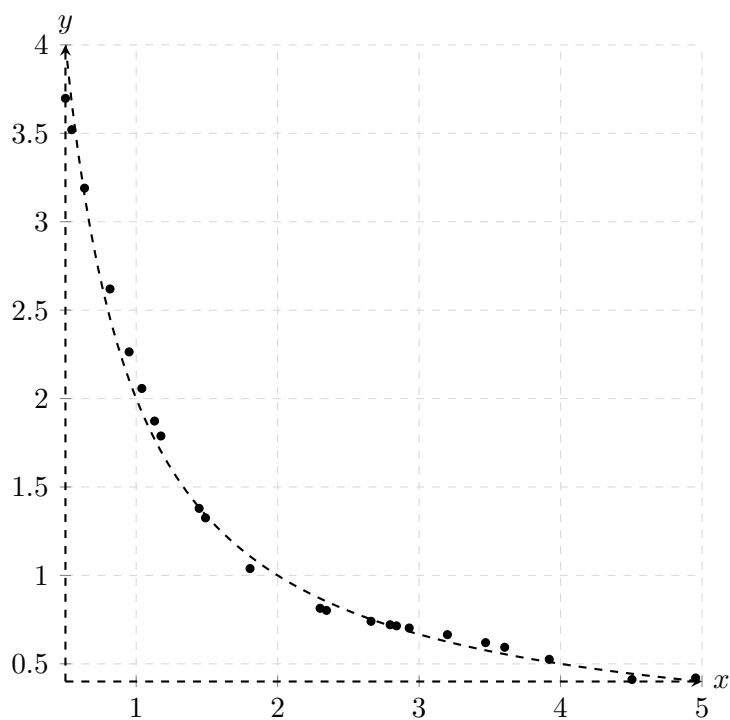


Рис. 2 — Интерполяция  $f(x) = \frac{2}{x}$  на  $[0.5, 5]$ .

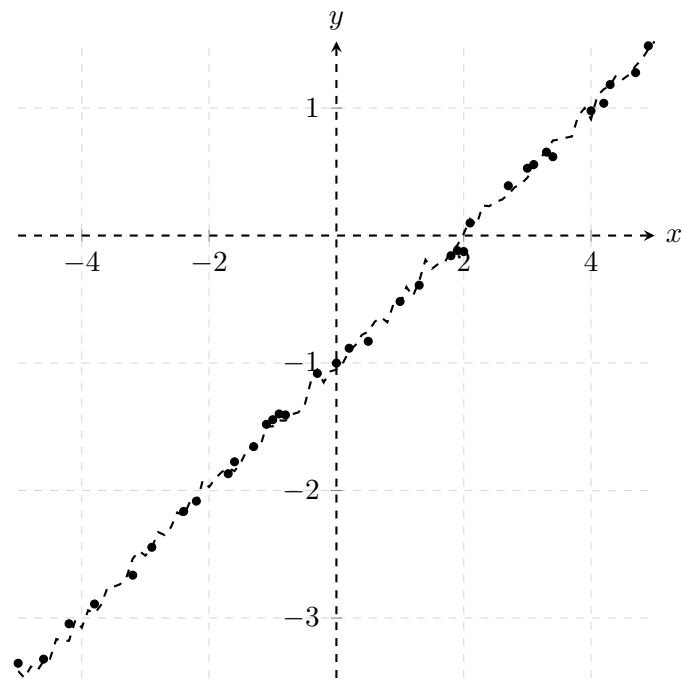


Рис. 3 — Интерполяция  $f(x) = 0.5x - 1$  на  $[-5, 5]$  с добавлением шума.

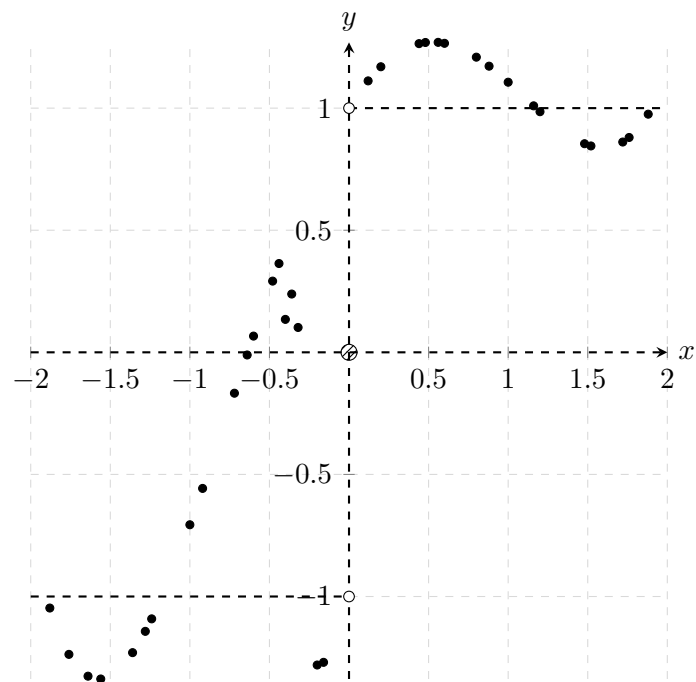


Рис. 4 — Интерполяция  $f(x) = \operatorname{sgn} x$  на  $[-2, 2]$ .

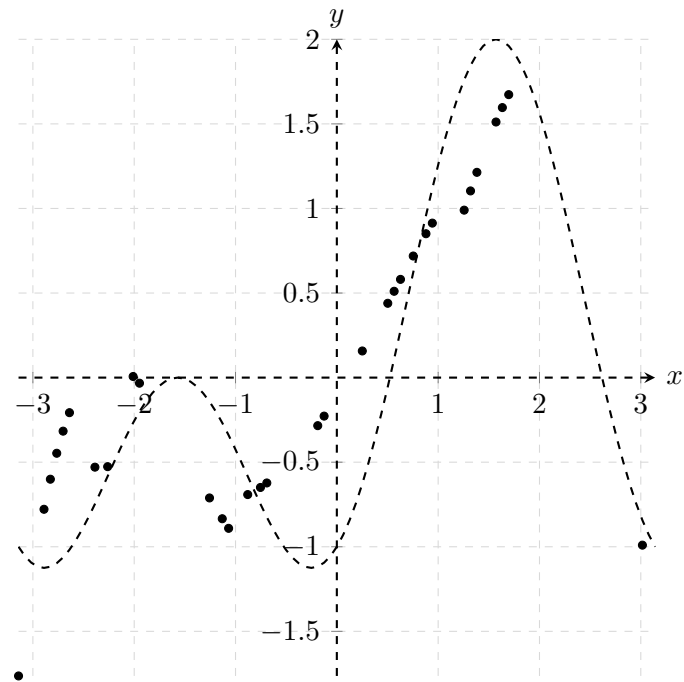


Рис. 5 — Интерполяция  $f(x) = \sin x - \cos 2x$  на  $[-\pi, \pi]$ .

Обратите внимание, что для каждого изображённого  $x$  строился свой интерполяционный многочлен в форме Лагранжа. Они строились за разное количество итераций (*до достижения точки останова*), поэтому соединять их в один график нет смысла.

### 3 Заключение

В ходе выполнения лабораторной работы у меня возникали некоторые вопросы, на которые я смог найти ответ самостоятельно или в различных источниках. Поэтому приведу ответы на них здесь в качестве результата.

**Почему мы не стали «честно» решать СЛАУ?** Действительно, можно было бы пойти таким путём. Но в таком случае матрица *плохо обусловлена*:

$$\text{cond}(A) = \frac{M}{m} \gg 1, \quad M = \max_{|x|=1} |Ax|, \quad m = \min_{|x|=1} |Ax|$$

Это значит, что при малых изменениях входных данных решение системы изменяется сильно. Мы не можем это игнорировать, потому что вычисления на процессоре не всегда точные и содержат погрешность округления.

**Почему мы используем корни многочлена Чебышёва?** Конечно, в качестве интерполяционных узлов мы можем выбрать любые точки из области определения интерполируемой функции. Но должна быть причина, по которой мы остановились на функции такого странного вида.

В профильной литературе доказывается, что отклонение  $f(x)$  от  $L_n(x)$  пропорционально величине

$$\sup_{[a,b]} |\omega_n(x)| = \sup_{[a,b]} \left| \sum_{i=1}^n x - x_i \right|,$$

где  $\{x_i\}_{i=1}^n$  — интерполяционные узлы.

Положим  $\omega_n(t) = 2^{1-n}T_n(t)$ , тогда отклонение от нуля окажется не больше  $2^{1-n}$ . И оказывается, что это *наименьший результат из возможных*.

Допустим следующее:

$$\exists P_n(t), a_n = 1: \sup_{[a,b]} |P_n(t)| < 2^{1-n}$$

Тогда многочлен  $2^{1-n}T_n(t) - P_n(t)$  степени  $n - 1$  имеет  $n + 1$  точку с поочерёдно положительными и отрицательными знаками:

$$x_m = \frac{m\pi}{n}, \quad m = \overline{0, n}$$

Значит, многочлен имеет  $n$  корней по теореме Больцано–Коши, что противоречит основной теореме алгебры.  $\square$

**Какие преимущества у формы Лагранжа?** Как видно из формулы, при неизменной сетке значение интерполяционного многочлена простым образом зависит от интерполируемой функции  $f$ . Этим можно воспользоваться, если мы хотим поменять функцию  $f$ , но оставить предыдущую сетку: достаточно пересчитать значения функции  $f$  в интерполяционных узлах.

Если мы воспользуемся интерполяционным многочленом *в форме Ньютона*, то при смене функции необходимо пересчитать таблицу разделённых разностей.

**Какие недостатки у формы Лагранжа?** Как видно из блок-схемы, когда мы добавляем новую точку в сетку, происходит повышение степени многочлена. Причём значение интерполирующей функции в точке мы вынуждены пересчитать с нуля. Мы не можем сделать это рекуррентно, опираясь на предыдущий результат (*что было бы полезно, учитывая итерационную природу алгоритма*).

Интерполяционный многочлен в форме Ньютона решает эту проблему: достаточно посчитать значение некоторого одночлена в точке и сложить с прошлым значением функции.

**Какая скорость сходимости у метода?** В качестве точки останова в методе используется условие  $|L_n(x) - L_{n-1}(x)| < 0.01$ . Оценим скорость сходимости снизу.

Пусть  $e_n = f(x) - L_n(x)$  — мы уже знаем, что оно не превосходит по модулю  $2^{1-n}$  на отрезке  $[a, b]$ . Имеем:

$$|L_n(x) - L_{n-1}(x)| = |(f(x) - L_{n-1}(x)) - (f(x) - L_n(x))| = |e_{n-1} - e_n|$$

То есть скорость сходимости нашего метода можно оценить сверху скоростью сходимости  $L_n(x)$  к  $f(x)$ :

$$|L_n(x) - L_{n-1}(x)| \leq |e_{n-1}| + |e_n| \leq \sup_{[a,b]} (|e_{n-1}| + |e_n|) = 2^{2-n} + 2^{1-n} = 6 \cdot 2^{-n}$$

Получаем линейную сходимость как нижнюю оценку.

**Какая асимптотическая сложность у итерации?** Глядя на блок-схему, построение интерполяционного многочлена в форме Лагранжа происходит за  $\mathcal{O}(n)$ , при этом используется  $\mathcal{O}(1)$  памяти. Поиск корней многочлена Чебышёва также происходит за  $\mathcal{O}(n)$ , используя ту же  $\mathcal{O}(1)$  память.

На каждой итерации алгоритма выделяется  $\mathcal{O}(n)$  памяти под массив корней многочлена Чебышёва и последовательно выполняются две упомянутые функции. Итого имеем:

- $\mathcal{O}(n)$  — временная асимптотическая сложность итерации;
- $\mathcal{O}(n)$  — пространственная асимптотическая сложность итерации.

**Почему получилась большая невязка?** Можно заметить, что на рис. 5 интерполирующая функция обладает высокой невязкой и мало чем походит на исходную функцию. Предполагаю, что это связано с выбором точки останова.

Условие  $|L_n(x) - L_{n-1}(x)| < 0.01$  проверяет сходимость разности интерполирующих многочленов к нулю. Но в общем случае это не должно гарантировать сходимость разности  $|f(x) - L_n(x)|$  к нулю, поэтому возможна ложная остановка итераций. Чтобы повысить точность многочлена, лучше его начинать строить на сетке с большим числом узлов.

На рис. 4 изображён график функции с разрывом в точке  $x = 0$ . Здесь ситуация иная: выбранный отрезок содержит точку разрыва. В её окрестности поведения функции сложно описать непрерывным многочленом, поэтому неизбежно возникает общая высокая невязка.

## 4 Источники литературы

- [1] Петров, И. Б. Лекции по вычислительной математике : учебное пособие / И. Б. Петров, А. И. Лобанов. — М. : Интернет-Университет Информационных Технологий ; БИНОМ. Лаборатория знаний, 2006. — 523 с. : ил., табл. — (Серия «Основы информационных технологий»).
- [2] Березин, И. С. Методы вычислений : в 2 т. / И. С. Березин, Н. П. Жидков. — 2-е изд., стереотип. — М. : Гос. изд-во физ.-мат. лит., 1962. — Т. 1. — 464 с.